Global Solution 2026.2

Cognitive Cybersecurity

Alerta Criadouro

Matheus Rebelo - RM 570260
Felipe Olmos - RM 570946
Luiz Guilherme - RM 573486

1. A solução que avaliamos

Neste trabalho atuamos como uma consultoria de segurança contratada para avaliar o protótipo Alerta Criadouro, o mesmo sistema das outras matérias da Global Solution.

- Problema: cerca de 75% dos focos do mosquito da dengue ficam dentro das casas (governo de SP, via Poder360, fev/2026). Em 2024 o Brasil teve 6,48 milhões de casos prováveis e 5.972 mortes (Agência Brasil). A larva vira mosquito em uns 10 dias, e a família só percebe quando alguém adoece.
- Público (único): famílias que moram em casas com quintal em bairros da cidade de São Paulo.
- Solução: um aparelho com ESP32 na área de serviço observa um potinho sentinela no quintal. O DHT22 mede temperatura e umidade e o HC-SR04 mede se há água no potinho e há quantos dias. LEDs e buzzer avisam a família (OK, ATENÇÃO, ALERTA) e o aparelho manda um JSON para uma API em Flask, que grava num arquivo CSV.
- Uso de IA: um modelo de Machine Learning diz o nível de risco de dengue da semana na cidade e uma rede LSTM prevê os casos das próximas semanas, os dois treinados com dados reais do InfoDengue (Fiocruz).
- Dados usados: leituras do quintal (temperatura, umidade, distância, dias com água), identificador do aparelho, localização aproximada (bairro), IP de quem envia e, no futuro app, o contato da família.

Arquitetura simplificada

Usuário	Aplicação	API	Banco de dados	Modelo de IA
Família (vê o LED e, no futuro, o app); agente de saúde (consulta os alertas)	ESP32 no quintal (firmware) e, no futuro, app da família	Flask: POST /leituras, GET /leituras, GET /indicadores (no protótipo roda no Colab com túnel público)	leituras.csv no protótipo; PostgreSQL (RDS) na versão de nuvem	Random Forest / XGBoost (risco da semana) e LSTM (casos), treinados com o InfoDengue

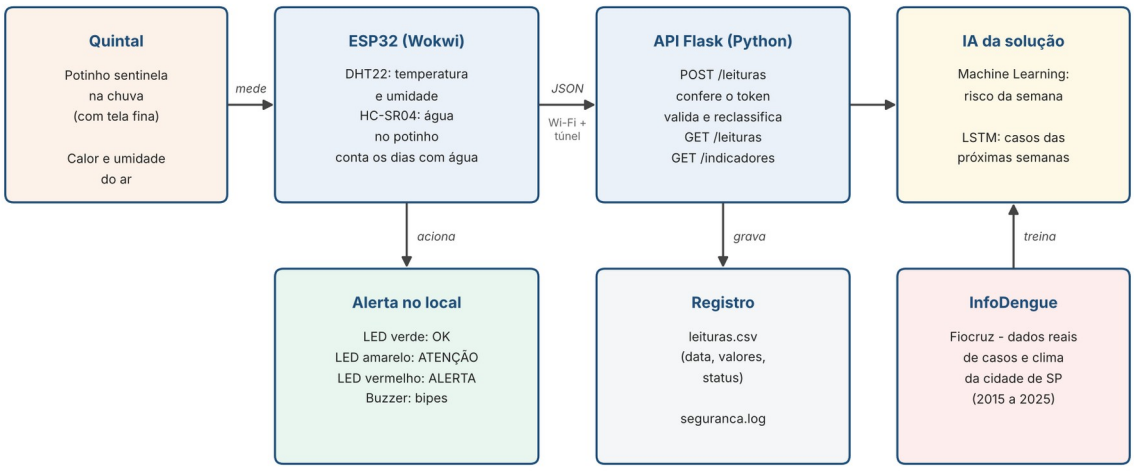


Figura 1 - Arquitetura do Alerta Criadouro

Sobre o armazenamento: o protótipo grava em CSV. Para esta análise tratamos o CSV como o banco de dados do sistema, porque ele guarda os mesmos dados e corre os mesmos riscos (vazamento, alteração, exposição). A evolução natural é um banco real, o PostgreSQL gerenciado (Amazon RDS), que já usamos na matéria de Cloud.

2. Superfície de ataque

Olhamos cada parte do sistema e perguntamos: por onde alguém poderia atacar?

Parte	Ameaças e vulnerabilidades encontradas
Usuários	Família cai em golpe usando o nome do projeto (falso agente de saúde, WhatsApp falso). Agente de saúde com senha fraca no futuro painel. Alguém da casa mexe no aparelho ou tampa o potinho.
Aplicação (ESP32)	O token e o endereço da API ficam gravados no firmware: quem tiver o aparelho na mão pode copiar. No simulador o envio é por http, sem criptografia. Firmware sem atualização segura.
API	Abuso de requisições (derrubar a API ou encher o CSV). Envio de leituras falsas por quem não é o aparelho. JSON malformado ou com valores absurdos para quebrar a API. Consulta dos dados por qualquer pessoa.
Banco de dados (CSV)	Vazamento do arquivo com a rotina da casa e o bairro. CSV injection: um dispositivo_id com fórmula do Excel vira um link malicioso quando o agente abre o arquivo. Perda do arquivo sem backup.
Modelo de IA	Envenenamento dos dados (data poisoning): leituras falsas ou base do InfoDengue adulterada fazem o modelo aprender errado e dar risco baixo numa semana de surto. Troca do arquivo do modelo treinado.

3. Matriz de riscos

Classificamos cada risco por impacto (o tamanho do estrago) e probabilidade (a chance de acontecer no nosso contexto). O nível vem da combinação dos dois.

#	Risco	Impacto	Probab.	Nível
R1	Token do aparelho copiado e usado para mandar leituras falsas	Alto	Média	Alto
R2	Abuso de requisições (rajada de envios) derrubando a API	Médio	Alta	Alto
R3	Leituras falsas fazendo o aparelho mostrar OK quando há criadouro	Alto	Média	Alto
R4	Vazamento do CSV com bairro e rotina da casa	Alto	Média	Alto
R5	CSV injection pelo campo dispositivo_id	Médio	Média	Médio
R6	Interceptação dos dados em trânsito (http no simulador)	Médio	Média	Médio
R7	Base do InfoDengue adulterada envenenando o modelo de IA	Alto	Baixa	Médio
R8	Golpe do falso agente de saúde usando a dengue	Alto	Média	Alto
R9	Acesso indevido ao futuro painel por senha fraca	Médio	Média	Médio

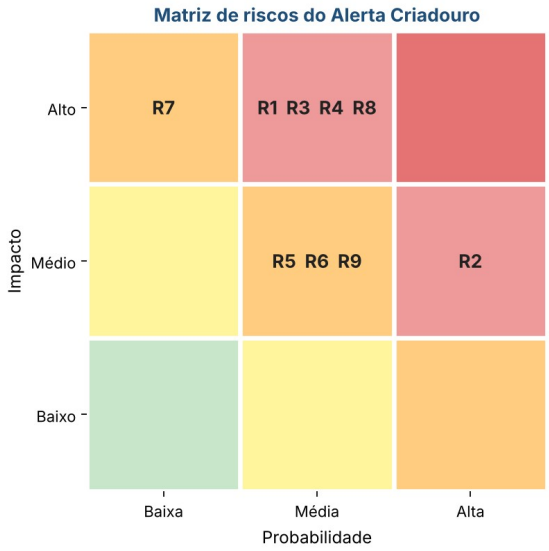


Figura 2 - Mapa de calor dos riscos

Os riscos mais altos são os que atingem a confiança no alerta (R1 e R3), a privacidade da família (R4), a disponibilidade da API (R2) e a segurança física das pessoas (R8). Um alerta falso de OK é perigoso porque a família deixa de vistoriar o quintal justamente quando precisava.

4. Controles de segurança

Risco	Controle	Situação
R1, R3	Token do aparelho no cabeçalho X-Token-Dispositivo, guardado em variável de ambiente e conferido em toda requisição (401 se errado). Na versão final, um token por aparelho, trocado de tempos em tempos.	Implementado (token único no protótipo)
R2	Rate limiting: no máximo 20 envios por minuto por IP; acima disso a API responde 429.	Implementado
R3, R5	Validação do JSON: campos obrigatórios, tipos, faixas físicas dos sensores e formato do dispositivo_id. A API recalcula a classificação e não confia no status enviado.	Implementado
Todos	Registro de logs em seguranca.log: ACEITO, RECUSADO e BLOQUEADO, com IP e motivo.	Implementado
R7	Hash SHA-256 da base do InfoDengue conferido pelos notebooks de IA antes de treinar.	Implementado
R4, R9	GET /leituras e /indicadores também exigem token. No futuro: login com MFA para agentes e controle de acesso por perfil (família vê só a sua casa).	Parcial
R6	HTTPS no aparelho real (no Wokwi o túnel já é https na ponta pública, mas o ESP32 simulado usa http).	Proposto
R4	Criptografia em repouso (RDS com criptografia ligada) e backup diário automático do banco.	Proposto
R2	Firewall/WAF na frente da API (Cloudflare ou AWS WAF) para barrar tráfego malicioso antes da API.	Proposto
R8	Campanha de conscientização e canal oficial de confirmação (detalhado na seção 7).	Proposto

Justificativa técnica das escolhas

Escolhemos implementar as proteções dentro da própria API porque é por ela que entram todos os dados, tanto os que acendem o alerta quanto os que vão para a IA. Proteger a porta de entrada cobre vários riscos de uma vez.

- Token em vez de login com senha: o ESP32 não tem teclado nem pessoa para digitar senha. Um token fixo por aparelho, enviado no cabeçalho, é o jeito mais simples e comum de autenticar dispositivos IoT.
- Rate limiting por IP com janela de 60 segundos: o aparelho manda uma leitura a cada 5 segundos (12 por minuto). Com limite de 20 sobra folga para o uso normal, e uma rajada de ataque é cortada rápido. Usamos uma fila (deque) por IP, sem depender de biblioteca externa.
- Validação com faixas físicas: temperatura entre -10 e 60 °C, umidade entre 0 e 100% e distância entre 0 e 400 cm (alcance do HC-SR04). Valores fora disso não existem no quintal, então são sinal de erro ou ataque.
- Reclassificar no servidor: se a API aceitasse o status que vem do aparelho, bastaria mandar status OK para esconder um criadouro. Recalculando, o resultado só depende dos valores medidos.
- Logs com motivo: sem log não dá para saber que alguém tentou atacar. O log mostra quem foi recusado e por quê, sem gravar dados pessoais além do IP.
- Hash SHA-256: é uma forma barata de garantir que a base usada no treino é a mesma que baixamos da Fiocruz.

5. Demonstração: proteções funcionando

Todas as proteções abaixo estão no arquivo api/app.py e foram testadas no dia 05/11/2026, com a API rodando no Google Colab e acessada pela internet (túnel https://vsnet-adequate-spaces-era.trycloudflare.com).

5.1 Token do dispositivo (autenticação)

```
def token_valido():
    token = request.headers.get("X-Token-Dispositivo")
    return bool(TOKEN_DISPOSITIVO) and token == TOKEN_DISPOSITIVO

# o token não fica no código: vem de variável de ambiente
TOKEN_DISPOSITIVO = os.environ.get("TOKEN_DISPOSITIVO")
```

Evidência: uma requisição com o token errado chute-123 e outra sem token receberam HTTP 401, e o ESP32 do Wokwi, com o token certo, recebeu HTTP 201 em todas as leituras.

```
token errado: 401
Teste extra - sem token do dispositivo
HTTP 401
Resposta: {"erro": "token do dispositivo ausente ou invalido"}
```

5.2 Rate limiting

```
def passou_do_limite(ip):
    agora = time.time()
    fila = requisicoes_por_ip[ip]
    while fila and agora - fila[0] > 60:
        fila.popleft()
    if len(fila) >= LIMITE_POR_MINUTO:
        return True
    fila.append(agora)
    return False
```

Evidência: mandamos 25 requisições seguidas. As 19 primeiras foram recusadas pelo token (401) e, quando o IP passou de 20 requisições no minuto, as seguintes foram bloqueadas com 429, antes mesmo de a API olhar o token.

```
rajada de 25 envios: [401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 401,
```

```
401, 401, 401, 401, 429, 429, 429, 429, 429, 429]
```

5.3 Registro de logs

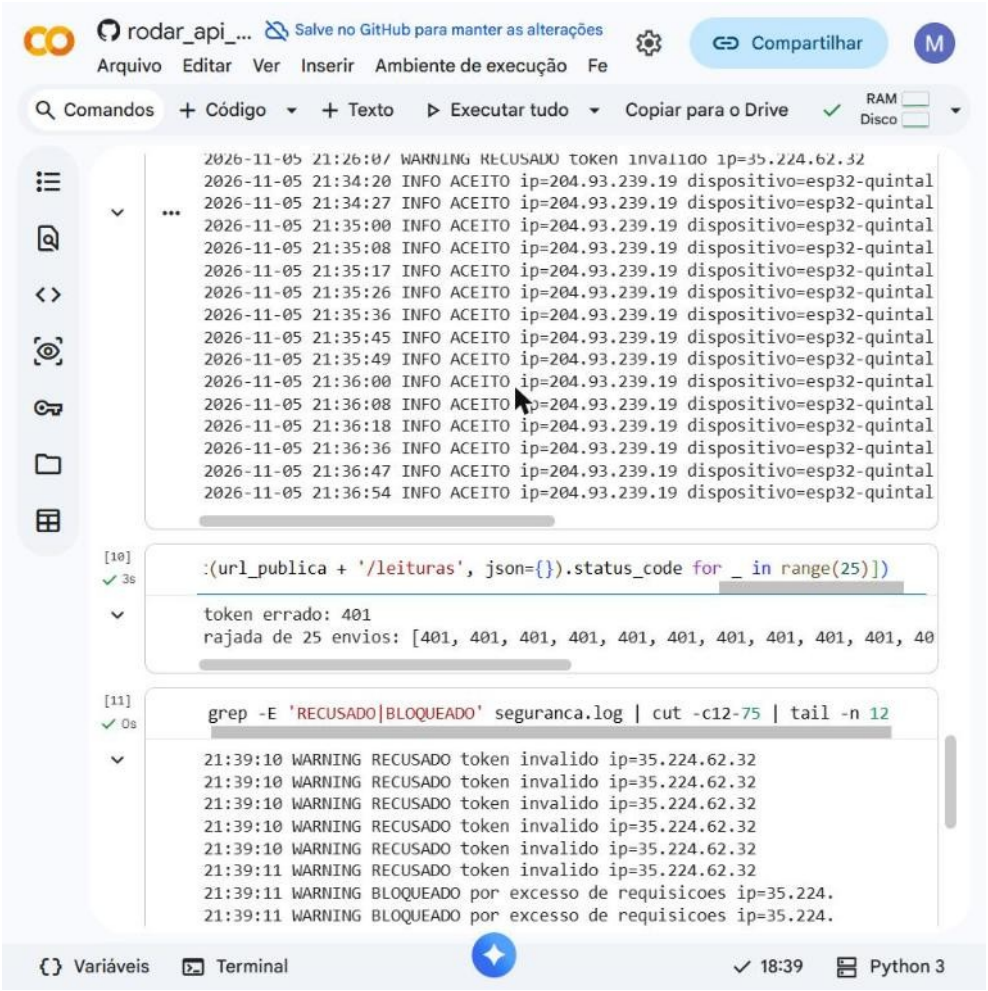


Figura 3 - seguranca.log com ACEITO, RECUSADO e BLOQUEADO

```
21:39:11 WARNING RECUSADO token invalido ip=35.224.62.32
21:39:11 WARNING BLOQUEADO por excesso de requisicoes ip=35.224.62.32
21:36:36 INFO ACEITO ip=204.93.239.19 dispositivo=esp32-quintal-001 dias=3 temp=29 umid=78
status=ALERTA
```

Nessa execução o horário do log saiu em UTC (3 horas a mais que São Paulo). Ajustamos o código para gravar no horário de São Paulo, como mostra o teste da seção 5.4.

5.4 Validação do JSON e correção do CSV injection

Durante a análise achamos uma falha real: o dispositivo_id era gravado no CSV sem filtro. Corrigimos com uma regra que só aceita letras minúsculas, números e hífen, e testamos de novo (execução local, 05/11/2026 18:45):

```
PADRAO_ID = re.compile(r"^[a-z0-9-]{3,40}$")

POST /leituras {"dispositivo_id": "=HYPERLINK(\"http://site-falso.com\",\"Clique aqui\")", ...}
-> 400 {"detalhes":["dispositivo_id invalido (use so letras minusculas, numeros e hifen)"],
        "erro":"leitura recusada"}

POST /leituras {"dispositivo_id": "esp32-quintal-001", "valor": 2, ...}
-> 201 {"status":"ATENCAO","nivel_risco":"atencao", ...}

seguranca.log:
```

```
2026-11-05 18:45:56 WARNING RECUSADO dados invalidos ip=127.0.0.1 erros=['dispositivo_id
invalido ...']
2026-11-05 18:45:56 INFO ACEITO ip=127.0.0.1 dispositivo=esp32-quintal-001 dias=2 temp=28 umid=80
status=ATENCAO
```

A validação também barrou, pela internet, uma leitura sem o campo valor e com 150 °C:

```
HTTP 400
{"detalhes": ["campo obrigatorio ausente: valor", "temperatura_c fora da faixa (-10 a 60)"],
 "erro": "leitura recusada"}
```

5.5 Hash de integridade da base de IA

```
import hashlib
HASH_ESPERADO = "741ccfb2a01cf5d98614fcffd4ddfaf41a748b4b0549211435879c1bec97a03b"
hash_atual = hashlib.sha256(open("infodengue_sp_2015_2025.csv", "rb").read()).hexdigest()
assert hash_atual == HASH_ESPERADO, "Base alterada! Treino cancelado."
```

Os notebooks de Machine Learning e de Redes Neurais rodam esse teste antes de treinar.

6. Privacidade e LGPD

Quais dados pessoais são tratados?

- Identificador do aparelho (esp32-quintal-001), que fica ligado a uma casa e, portanto, a uma família.
- Localização aproximada da casa (latitude e longitude com 3 casas decimais, o que mostra o bairro).
- Endereço IP da conexão, gravado no log.
- Rotina da casa: os horários em que o potinho enche e esvazia mostram quando a família está em casa.
- Na versão com app: nome e telefone de contato do responsável, se ele quiser receber o alerta no celular.

Quais dados sensíveis podem existir?

Pela LGPD (art. 5°, II), dado de saúde é dado sensível. O sistema não pede, mas pode acabar recebendo: o campo relato (texto livre da família) pode trazer algo como "meu filho está com dengue", e o formulário de validação do projeto pergunta se a família já teve dengue em casa. Por isso esses dados recebem cuidado extra.

Quais proteções serão aplicadas?

- Minimização: o aparelho manda só o necessário. A localização tem 3 casas decimais (bairro, não a casa). O protótipo não guarda nome, endereço nem telefone.
- Base legal: consentimento da família na instalação (art. 7°, I). Para o relato e o formulário, que podem ter dado de saúde, consentimento específico e destacado (art. 11, I).
- Formulário de validação anônimo: não pede nome nem endereço, só o bairro e respostas sim ou não.
- Acesso restrito: a consulta aos dados exige token. Na versão final, cada família vê só a sua casa e o agente de saúde vê dados juntados por bairro.
- Segurança: HTTPS em trânsito, criptografia no banco, backup e logs (seção 4).
- Prazo de guarda: as leituras são apagadas depois de 12 meses. Para estudo, só ficam números somados por bairro.

- Direitos do titular (art. 18): a família pode pedir para ver, corrigir ou apagar seus dados a qualquer momento, e pode desligar o envio e usar só o alerta local.

Como o usuário será informado?

Na instalação, a família recebe um termo de uma página, em linguagem simples, dizendo o que o aparelho mede, o que vai para a internet, quem vê os dados e como pedir para apagar. A caixa do aparelho tem um adesivo com um QR code para a política de privacidade e um contato do responsável pelos dados. No app, um aviso aparece antes de qualquer campo de texto livre lembrando que não é preciso escrever informação de saúde.

7. Engenharia social: o golpe do falso agente de saúde

Cenário de ataque

Golpistas sabem que, em época de dengue, as famílias abrem a porta para quem diz vir fazer a vistoria do quintal. Com o Alerta Criadouro, o golpe pode ficar mais convincente:

- A família recebe no WhatsApp uma mensagem com o nome do projeto: "Seu Alerta Criadouro detectou foco do mosquito. Um agente vai visitar sua casa hoje. Confirme no link". O link pede nome, endereço e CPF, ou cobra uma "taxa de vistoria" por Pix.
- Pouco depois, uma pessoa com colete parecido com o da prefeitura bate na porta dizendo que o aparelho mandou o alerta. Lá dentro, ela furta objetos ou observa a casa para um roubo depois.
- Como o LED vermelho estava mesmo aceso, a história parece verdadeira. Os mais expostos são idosos que ficam sozinhos em casa durante o dia.

Contramedidas

- Regra clara, repetida em tudo: o Alerta Criadouro nunca manda link, nunca cobra nada e nunca marca visita. Essa frase vai no termo de instalação, no adesivo da caixa e no app.
- Confirmar antes de abrir a porta: o agente de combate a endemias de verdade é da prefeitura, usa uniforme e crachá com nome e matrícula, e a família pode confirmar ligando no 156 (Central de Atendimento da Prefeitura de São Paulo). Na dúvida, não deixa entrar.
- Nada de dado pessoal por mensagem: nenhuma comunicação oficial pede CPF, senha ou pagamento.
- Conscientização: um cartão ilustrado com 3 passos (desconfie, confirme no 156, não pague nada), entregue na instalação e pensado também para idosos.
- Canal de denúncia: a família avisa pelo app se recebeu mensagem falsa, e a equipe alerta as outras casas do bairro.
- Para a equipe: MFA na conta de quem administra o sistema, para que um golpista não consiga se passar pela equipe usando uma senha roubada.

8. Conclusão

O maior perigo do Alerta Criadouro não é alguém roubar o aparelho, e sim alguém fazer a família desconfiar do alerta ou usar o projeto para enganar as pessoas. Por isso protegemos primeiro a porta de entrada dos dados (token, limite de requisições, validação e logs, todos funcionando), garantimos que a IA aprende com a base certa (hash) e tratamos a privacidade desde o começo, guardando só o

bairro. Os próximos passos são HTTPS no aparelho real, um token por casa, banco com criptografia e backup, e a campanha contra o golpe do falso agente.

Referências

- Brasil. Lei nº 13.709/2018 - Lei Geral de Proteção de Dados Pessoais (LGPD).
- OWASP. API Security Top 10 (2023) e CSV Injection. https://owasp.org
- Poder360. 75% dos focos do mosquito da dengue estão dentro das casas, diz governo de SP. Fevereiro de 2026.
- Agência Brasil. Brasil registra 6,48 milhões de casos prováveis de dengue em 2024.
- Prefeitura de São Paulo. Central SP156.
- Código do projeto: https://github.com/matheusrebelo04-maker/alerta-criadouro (pasta api)